Multiple Inheritance in Java

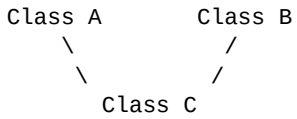
Java Programming Lab Record — Multiple Test Cases

1. Aim

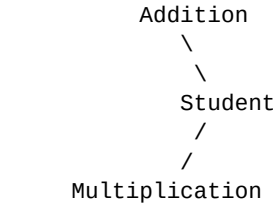
To implement Multiple Inheritance in Java using interfaces, execute the program using multiple test cases, and verify the results.

2. Concept

Multiple inheritance means a single class inherits features from more than one parent. Java does not support multiple inheritance using classes:



Instead, Java achieves multiple inheritance using interfaces.



Here, Student implements both Addition and Multiplication.

3. Java Program

```
import java.util.Scanner;

// First interface
interface Addition {
    int add(int a, int b);
}

// Second interface
interface Multiplication {
    int multiply(int a, int b);
}

// Class implementing two interfaces
class Student implements Addition, Multiplication {

    // Implementing Addition interface method
    public int add(int a, int b) {
        return a + b;
    }

    // Implementing Multiplication interface method
    public int multiply(int a, int b) {
        return a * b;
    }

    void displayResult(int a, int b) {
        System.out.println("First Number: " + a);
        System.out.println("Second Number: " + b);
        System.out.println("Addition: " + add(a, b));
        System.out.println("Multiplication: " + multiply(a, b));
    }
```

```
}

// Main class
public class MultipleInheritance {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of test cases: ");
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {

            System.out.println("\n----- Test Case " + i + " -----");

            System.out.print("Enter first number: ");
            int a = sc.nextInt();

            System.out.print("Enter second number: ");
            int b = sc.nextInt();

            // Creating object of Student class
            Student s = new Student();

            // Calling methods from both interfaces
            s.displayResult(a, b);
        }

        sc.close();
    }
}
```

4. Explanation

• **Addition** and **Multiplication** are two separate interfaces, each declaring one method.

• **Student** implements both interfaces at once with `implements Addition, Multiplication` — something Java does not allow with `extends` on two classes.

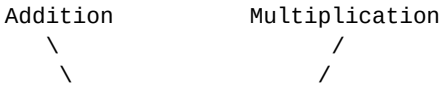
• Student provides concrete bodies for both `add()` and `multiply()`, then uses them inside its own `displayResult()` method.

• The program reads the number of test cases first, then loops that many times, creating a fresh Student object and computing both results for each pair of numbers.

5. Test Case Verification

Test Case	1st No.	2nd No.	Exp. Add.	Actual	Exp. Mult.	Actual
1	10	5	15	15 ■	50	50 ■
2	65	45	110	110 ■	2925	2925 ■
3	87	90	177	177 ■	7830	7830 ■

6. Inheritance Structure



```
            \                /
             Student Class
```

The important statement is:

```
class Student implements Addition, Multiplication
```

This means Student gets the functionality defined by both interfaces.

7. Important Exam Point

Java does not support:

```
class C extends A, B   // ■ Not allowed
```

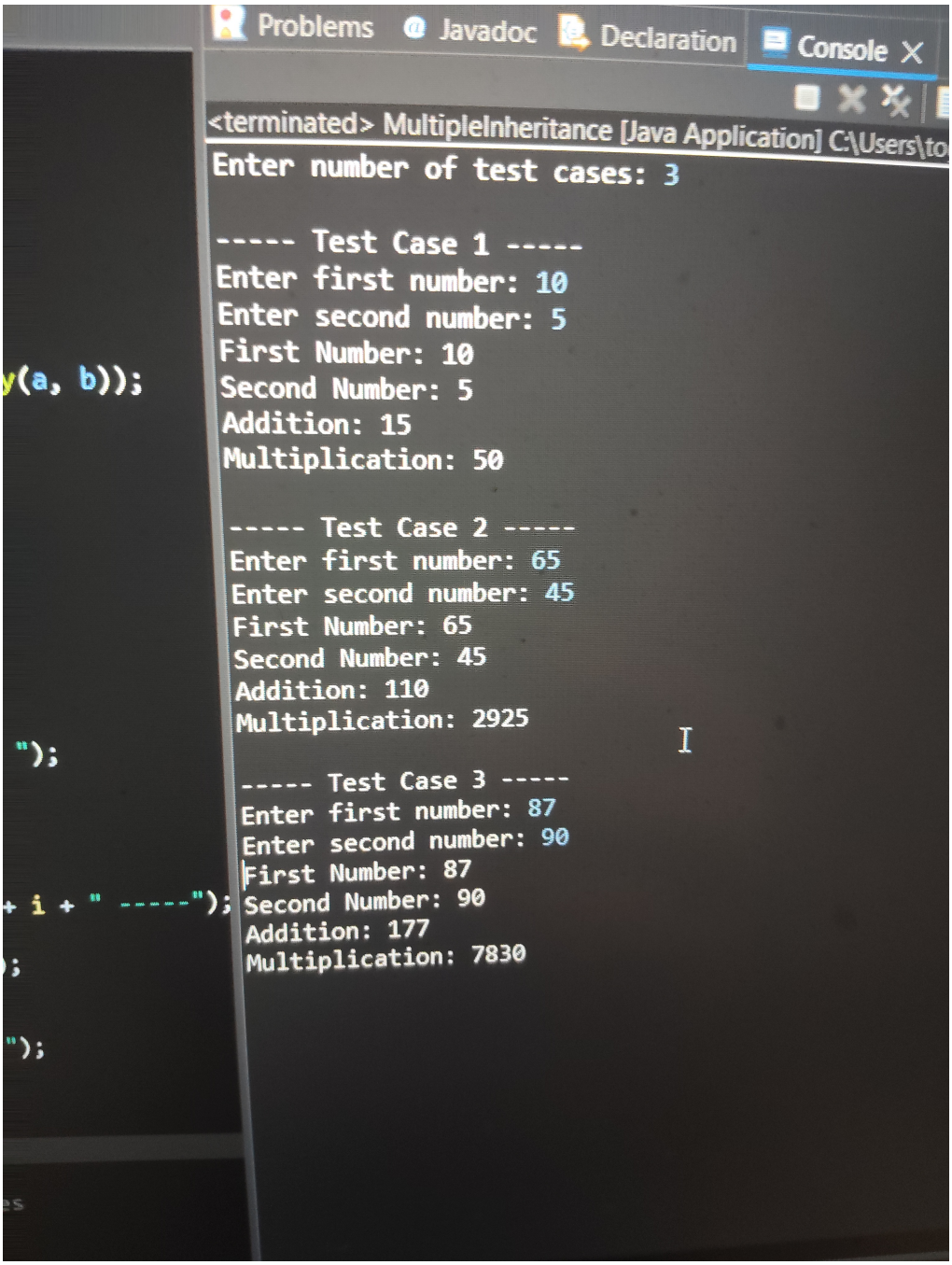
But Java supports:

```
class C implements A, B   // ■ Allowed
```

Therefore, multiple inheritance in Java is achieved through interfaces.

8. Output Screenshot

The screenshot below shows the program actually executed, running all three test cases in a single session:



The results verify that a single class can correctly implement and use methods from two different interfaces, confirming multiple inheritance in Java via interfaces works as expected.